



第02章 数值数据类型和运算

张仁斌@合肥工业大学软件学院

本章主要内容



2.1 编写简单的程序



2.2 标识符与变量、命名常量

2.3 赋值语句与赋值表达式

2.4 数值数据类型、运算及运算符优先级

2.5 数值类型转换

2.6 软件开发流程简介

2.7 常见错误

2.8 作业与实践



计算圆的面积

◆编写程序包括设计**算法**以及将算法转换为程序指令或代码

- 分析问题→设计解决方案→通过编程解决问题（简单问题也坚持这个过程）
- **程序= 算法+数据结构**（计算机科学家Nikiklaus Wirth, 1976）
- 算法是为了解决某个问题而采取的**方法**和执行**步骤**
- 算法可以用自然语言、伪代码或流程图等形式表示

◆计算圆面积的算法为：

- （1）读入圆的半径；
- （2）利用公式“面积=半径×半径× π ”计算面积；
- （3）输出圆的面积

【1】如何读入半径？

【2】在程序中如何保存半径？→**用变量**

IPO
(Input
Process
Output)

```
int main()
{
    // Step 1: Read in radius

    // Step 2: Compute area

    // Step 3: Display the area
}
```



lucid, systematic,
and penetrating
treatment of basic
and dynamic data
structures, sorting,
recursive algorithms,
language structures,
and compiling

NIKLAUS WIRTH

**Algorithms +
Data
Structures =
Programs**

变量是内存空间的一种抽象，程序中定义的每个变量在运行时都有与之对应的内存空间



◆ 程序执行期间，变量的值可改变；变量名尽量“**顾名思义**”（代码可读性）

◆ 须指定变量的数据类型，编译器按类型给变量分配**相应大小的内存**

➤ 常用变量数据类型：整数（int），浮点数（float），双精度浮点数（double）等

```
int main()
{
    double radius; // 定义变量
    double area;

    // Step 1: Read in radius

    // Step 2: Compute area

    // Step 3: Display the area
}
```

D:\Edu-CPP\chap02\ex02-01.exe

The area is 1256.64

```
#include <iostream>
using namespace std;
int main()
{
```

```
    double radius;
    double area;
```

编译时给变量分配内存，变量的值是不确定（随机）的（**调试状态查看其值**）

```
    // Step 1: Read in radius
```

```
    radius = 20;
```

给变量赋值后其值确定

```
    // Step 2: Compute area
```

```
    area = radius * radius * 3.14159;
```

先计算再赋值

```
    // Step 3: Display the area
```

```
    cout << "The area is " << area << endl;
```

```
    return 0;
```

```
}
```

输出变量的值，**不同于**`cout << "area"`



从键盘读取输入

◆从键盘读取输入，使程序可以接受用户的输入，程序更通用

■计算半径为输入任意正数的圆的面积

➤必须要有输入提示，否则，用户不知该干什么

标准控制台输入（“>>”指向变量），并将输入内容赋予变量（**cin**使程序进入等待转态，直至键盘输入并回车），可以用一条语句读取多个输入：**cin >> x >> y >> z**

C++支持变量随用随定义，尽量集中定义

```
#include <iostream>
using namespace std;

int main()
{
    // Step 1: Read in radius
    double radius;
    cout << "Enter a radius: "; // 输入提示
    cin >> radius;

    // Step 2: Compute area
    double area = radius * radius * 3.14159;

    // Step 3: Display the area
    cout << "The area is " << area << endl;

    return 0;
}
```

```
D:\Edu-CPP\chap02\ex02-02.exe
Enter a radius: |
D:\Edu-CPP\chap02\ex02-02.exe
Enter a radius: 20
The area is 1256.64
```



◆ 从键盘输入3个数，输出其平均值

I
P
O

```
#include <iostream>
using namespace std;
int main() {
    double number1, number2, number3;

    // Prompt the user to enter three numbers
    cout << "Enter three numbers: ";
    cin >> number1 >> number2 >> number3;

    // Compute average
    double average = (number1 + number2 + number3) / 3;

    // Display result
    cout << "The average of " << number1 << " " << number2
        << " " << number3 << " is " << average << endl;

    return 0;
}
```

一次性输入3个数据（空格分隔）

```
D:\Edu-CPP\chap02\ex02-03.exe
Enter three numbers: 1 2 3
The average of 1 2 3 is 2
```

分两次输入（敲两次回车键）

```
D:\Edu-CPP\chap02\ex02-03.exe
Enter three numbers: 1 2
3
The average of 1 2 3 is 2
```

分三次输入（敲三次回车键）

```
D:\Edu-CPP\chap02\ex02-03.exe
Enter three numbers: 1
2
3
The average of 1 2 3 is 2
```

本章主要内容



2.1 编写简单的程序

2.2 标识符与变量、命名常量



2.3 赋值语句与赋值表达式

2.4 数值数据类型、运算及运算符优先级

2.5 数值类型转换

2.6 软件开发流程简介

2.7 常见错误

2.8 作业与实践

标识符

代码片段中也偶尔使用i、j、k、x、y等简洁的变量名

◆标识符是由字母、数字和下划线组成的字符序列，用于变量、符号常量、函数、数组、文件等实体命名，应尽量“顾名思义”（代码可读性）

■标识符必须以字母或下划线开头，不能以数字开头，不能包含空格和运算符等

■标识符可以是任意长度，但C++编译器可能会施加一些限制，使用31个字符或更少的标识符以确保可移植性

■标识符不能是C++关键字/保留字（清单详见群文件）

➤关键词/保留字：系统中预定义的、语言或编译系统的实现中具有特殊含义的单词

➤常用关键词/保留字：

C/C++是大小写敏感的，因此，area、Area、AREA是不同的标识符

下面哪些标识符是正确的？哪些是C++关键字？

miles, Test, a++, --a, 4#R, \$4, x, apps, main, double, int, radius

if	else	unsigned	signed	while	union	this
int	char	double	throw	const	goto	virtual
for	float	break	auto	class	operator	case
do	long	typedef	static	friend	template	default
new	void	register	extern	return	enum	inline
try	short	continue	sizeof	switch	private	protected
asm	while	catch	delete	public	volatile	struct

变量



- ◆ 程序执行期间，其值**可改变**的量，用于暂存运算的中间结果或程序块之间的信息传递
- ◆ 变量定义告诉编译器依据**变量数据类型**分配适当大小的内存空间。定义一个变量的语法是：**数据类型 变量名**；
 - 例如：int count; // 定义一个整型变量count
 - double radius; // 定义一个双精度浮点型变量radius
 - 如果多个变量的数据类型相同，可以一起定义这些变量
 - 例如：int i, j, k;
- ◆ 按照惯例，变量名都是**小写的**，如果一个名称包含多个词（词之间不能用空格），则采用“**小驼峰**”法
 - 例如：double radius, **interestRate**;



◆ 尽量在定义变量时**初始化变量**（给变量赋初始值），避免编程错误

例如： `int count = 1;`

`int i = 1, j = 2;`

`int x(3), y(4);` // C++语法，等同 `int x, y; x = 3; y = 4;`

◆ 变量必须先定义，后使用

◆ 找出并纠正代码中的错误（**ex02-04**）：

```
#include <iostream>
using namespace std;

int Main()
{
    int i = k + 1;
    cout << I << endl;

    int i = 2;
    cout << i << endl;
    return 0;
}
```



命名常量

- ◆命名的常量是一个代表**固定值**的标识符
 - 程序执行期间，常量的值不可改变（改变常量的值，将报错）
- ◆定义常量的语法是：**const 数据类型 标识符名 = 常量值;**
例如：`const double PI = 3.14159;`
- ◆常量必须在定义时初始化
- ◆常量的名字习惯上用大写，例如**PI**，而不是**pi**或**Pi**
- ◆使用常量的优点
 - 不需反复输入同一值
 - 需修改常量的值时，例如将3.14159改为3.14，只需修改常量的定义
 - 顾名思义的常量名增加可读性



◆使用常量时，计算圆的面积的程序修改

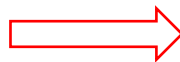
```
#include <iostream>
using namespace std;

int main()
{
    // Step 1: Read in radius
    double radius;
    cout << "Enter a radius: ";
    cin >> radius;

    // Step 2: Compute area
    double area = radius * radius * 3.14159;

    // Step 3: Display the area
    cout << "The area is " << area << endl;

    return 0;
}
```



```
#include <iostream>
using namespace std;

int main()
{
    const double PI = 3.14159;

    // Step 1: Read in radius
    double radius;
    cout << "Enter a radius: ";
    cin >> radius;

    // Step 2: Compute area
    double area = radius * radius * PI;

    // Step 3: Display the area
    cout << "The area is " << area << endl;

    return 0;
}
```

本章主要内容



2.1 编写简单的程序

2.2 标识符与变量、命名常量

2.3 赋值语句与赋值表达式



2.4 数值数据类型、运算及运算符优先级

2.5 数值类型转换

2.6 数值类型转换

2.7 常见错误

2.8 作业与实践



赋值语句与赋值表达式

- ◆ 定义一个变量之后，即可用赋值语句给变量赋值
 - 只能给变量赋值
- ◆ C/C++使用**等号**作为**赋值运算符**，赋值语句的语法为：**变量 = 表达式**；
 - 一个表达式表示一个运算，包含数值常量、变量和运算符，它们一起来求得一个结果值（该值赋给**等号左边**的变量），例如：

```
int y = 1;  
double radius = 1.0;  
int x = 5 * (3 / 2);  
x = y + 1;  
double area = radius * radius * 3.14159;
```

- 一个变量可以同时出现在一个赋值运算符的两边，例如

```
int x = 5 * (3 / 2);  
x = x + 1; // 将x + 1的计算结果赋给（更新）变量x
```



- ◆在C/C++中，一个赋值语句也可以当做一个表达式来处理，该表达式的值是赋值运算符左边变量的值，因此，赋值语句也可以作为一个赋值表达式。例如：

```
cout << x = 1;
```

■该语句相当于：

```
x = 1;
```

```
cout << x;
```

- ◆如果一个值要赋给多个变量，可以用下面的语句：

```
i = j = k = 2;
```

■相当于：

```
k = 2;
```

```
j = k;
```

```
i = j;
```

```
#include <iostream>
using namespace std;
int main()
{
    int i = j = k = 2; // 不能如此定义变量并初始化

    return 0;
}
```

本章主要内容



2.1 编写简单的程序

2.2 标识符与变量、命名常量

2.3 赋值语句与赋值表达式

2.4 数值数据类型、运算及运算符优先级



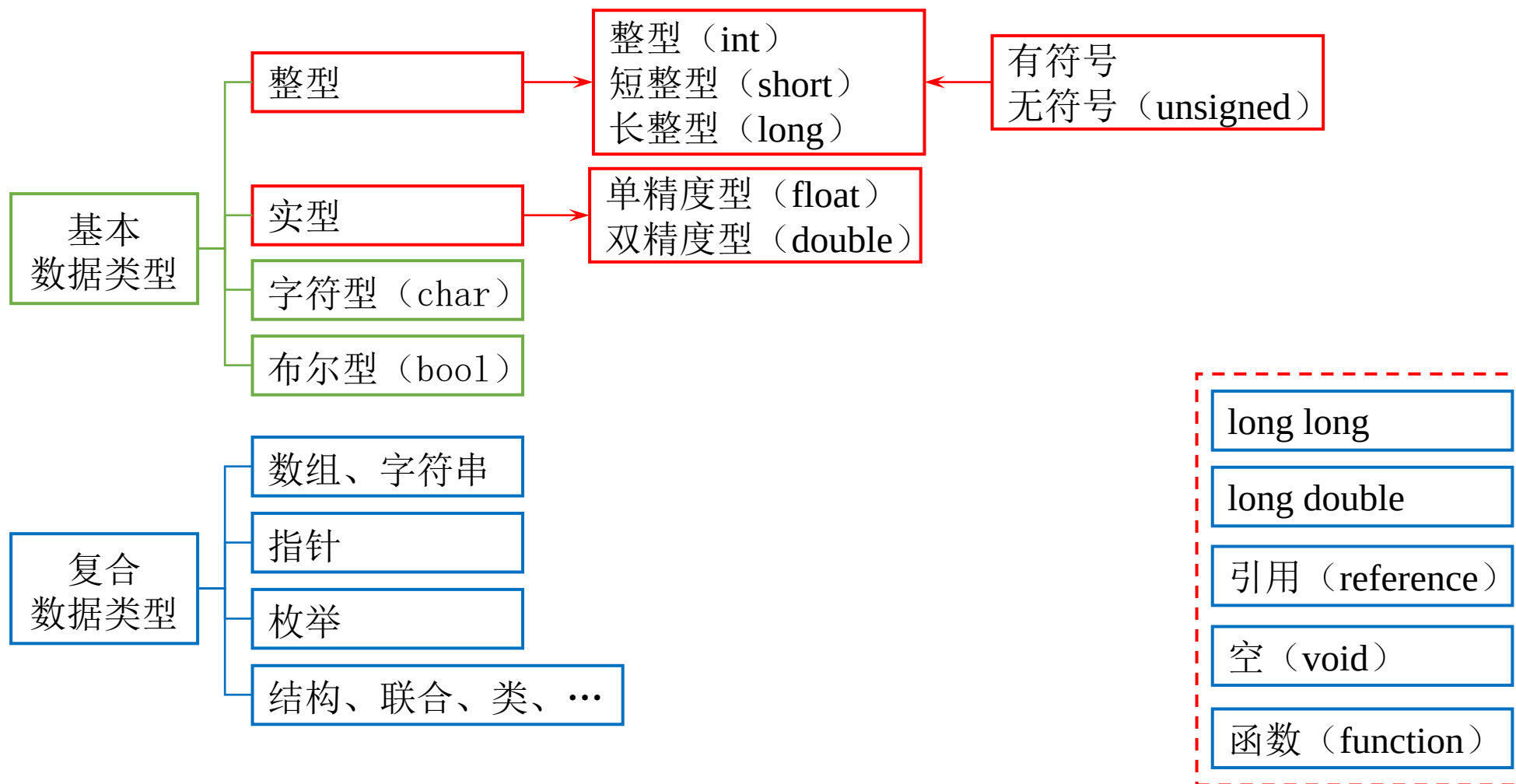
2.5 数值类型转换

2.6 软件开发流程简介

2.7 常见错误

2.8 作业与实践

C++数据类型



数值数据类型



◆ 每种数据类型都有其值域，C++标准未规定每种数据类型占用的内存大小（字节数），只规定大小顺序，具体长度由处理器和编译器确定

类型名	同义表示	值域	空间占用
short	short int	$-2^{15} \sim 2^{15}-1$ (-32,768~32,767)	16位有符号
unsigned short	unsigned short int	$0 \sim 2^{16}-1$ (65535)	16位无符号
int	signed	$-2^{31} \sim 2^{31}-1$	32位有符号
unsigned	unsigned int	$0 \sim 2^{32}-1$	32位无符号
long	long int	$-2^{31} \sim 2^{31}-1$	32位有符号
unsigned long	unsigned long int	$0 \sim 2^{32}-1$	32位无符号
long long		$-2^{63} \sim 2^{63}-1$	64位有符号
float		负数: $-3.4028235\text{E}+38 \sim -1.4\text{E}-45$; 正数: $1.4\text{E}-45 \sim 3.4028235\text{E}+38$	32位IEEE 754
double		负数: $-1.7976931348623157\text{E}+308 \sim -4.9\text{E}-324$; 正数: $4.9\text{E}-324 \sim 1.7976931348623157\text{E}+308$	32位IEEE 754
long double		负数: $-1.18\text{E}+4932 \sim -3.37\text{E}-4932$; 正数: $3.37\text{E}-4932 \sim 1.18\text{E}+4932$; 19位十进制有效数字	80位



◆用“类型名”和相应的“同义表示”定义变量的语句完全相同，例如：

short int i = 2; 等同 short i = 2;
unsigned int k = 3; 等同 unsigned k = 3;

◆**整数类型**短整型short、整型int和长整型long都分为有符号型（signed）和无符号型（unsigned）两类，所能表示的数值中，有符号型一半是负数，一半是正数，而**无符号型数值都是非负的**，因此存储在一个无符号整型数中的最大值，是一个有符号整型数所能保存的最大值的两倍

- 如果确定一个变量的值始终为非负，则将它定义为无符号数
- 扩展长整型为long long



- ◆ 浮点类型包括单精度浮点型float、双精度浮点型double和扩展双精度浮点型long double
- ◆ double占用的内存空间通常是float的两倍，double比float类型表示数据更精确

```
cout << "1.0 / 3.0 is " << 1.0 / 3.0 << endl;    // 1.0是双精度浮点数  
cout << "1.0F / 3.0F is " << 1.0F / 3.0F << endl; // 1.0F或1.0f是float类型
```

D:\Edu-CPP\chap02\ex02-06.exe

1.0 / 3.0 is 0.33333333333333333315

16个数字

1.0F / 3.0F is 0.333333343267440796

7个数字



◆数据类型的大小依赖于所使用的编译器和所使用的计算机

- 通常，int和long有相同的大小；在某些系统中，long需要8字节
- 可以使用**sizeof函数**查看一个**类型**或者**变量**在所使用的机器上占用的内存大小

```
#include <iostream>
using namespace std;
int main() {
    cout << "The size of int: " << sizeof(int) << " bytes" << endl;
    cout << "The size of long: " << sizeof(long) << " bytes" << endl;
    cout << "The size of double: " << sizeof(double) << " bytes" << endl;

    double area = 5.4;
    cout << "The size of variable area: " << sizeof(area) << " bytes" << endl;

    int age = 31;
    cout << "The size of variable age: " << sizeof(age) << " bytes" << endl;

    return 0;
}
```

D:\Edu-CPP\chap02\ex02-07.exe

```
The size of int: 4 bytes
The size of long: 4 bytes
The size of double: 8 bytes
The size of variable area: 8 bytes
The size of variable age: 4 bytes
```

数值文字常量



- ◆ 文字常量是指在程序中直接出现的常量值，例如下面语句中的“34”和“0.305”

```
int i = 34;
```

```
double footToMeters = 0.305;
```

必须记住:

- ①八进制常量使用前缀0
- ②十六进制常量使用前缀0x或0X

- ◆ 默认情况下，一个整数文字常量表示一个十进制整数，八进制整数文字常量使用前缀0，十六进制整数文字常量使用前缀0x或0X

例如: `cout << "0xFFFF: " << 0xFFFF << endl;`

`cout << "010: " << 010 << endl;`

- ◆ 浮点数可以采用特殊的语法书写科学记数法数字

例如, 1.23456×10^{-2} 可以写为 $1.23456\text{E}-2$

1.23456×10^2 可以写为 $1.23456\text{E}2$ 或 $1.23456\text{e}+2$

E (或e) 代表指数, 大小写均可

D:\Edu-CPP\chap02\ex02-08.exe

0xFFFF: 65535

010: 8

下面哪些是浮点数的正确写法?

12.3	12.3e+2	23.4e-2	-334.5
20.5	E-5	2E5.1	39



数值运算符

◆可作用于数值数据类型的运算符包括标准算术运算符

■运算符是实现各种运算的符号

■操作数是由运算符进行运算的值

■模运算（%）的用途很多

➤例如判断一个数是否是偶数（偶数%2得0）、是否整除

➤再如，如果今天是星期二，那么100天后是星期几？

运算	运算规则	示例
+	加	2 + 3
-	减	a - b
*	乘	2 * (-a)
/	除	12 / 3.0
%	取模	15 % 4
++	自增	a++, ++a
--	自减	b--, --b

若操作数均为整数，
运算结果为整数

操作数必须是整数

a的值为2

```
int a = 5 / 2;
```

```
float a = 5 / 2;
```

```
int a = 5.0 / 2.0;
```

a的值为2.5

```
float a = 5.0 / 2;
```

```
float a = 5 / 2.0;
```

```
float a = 5.0 / 2.0;
```

数值运算示例



◆ 将给定的秒数转换为分钟数和余下的秒数，例如，500秒是8分钟20秒

I
P
O

```
#include <iostream>
using namespace std;

int main()
{
    int seconds;
    cout << "Enter an integer for seconds: "; // Prompt the user for input
    cin >> seconds;
    int minutes = seconds / 60;
    int remainingSeconds = seconds % 60;
    cout << seconds << " seconds is " << minutes << " minutes and " << remainingSeconds << " seconds " << endl;

    return 0;
}
```

“拼接”输出内容，分析、理解如何“拼接”的

D:\Edu-CPP\chap02\ex02-09.exe

Enter an integer for seconds: 500
500 seconds is 8 minutes and 20 seconds



指数运算符

◆ **pow(a, b)**函数用于计算 a^b ，例如**pow(2.0, 3)**返回 2.0^3

■ pow是在cmath库文件中定义的函数，a和b是pow**函数的参数**，2.0和3是调用函数时实际传入的值

■ 某些C++编译器要求pow(a, b)函数的两个参数都是十进制

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    cout << "pow(2.0, 3) = " << pow(2.0, 3) << endl;
    cout << "pow(2.0, 2.0) = " << pow(2.0, 2.0) << endl;
    cout << "pow(9, 0.5) = " << pow(9, 0.5) << endl;
    cout << "pow(2.5, -2) = " << pow(2.5, -2) << endl;

    return 0;
}
```

编程尝试:

假设x和y是整数，写一个C++表达式计算 xy^2 ，并且得到的结果为浮点数

D:\Edu-CPP\chap02\ex02-10.exe

```
pow(2.0, 3) = 8
pow(2.0, 2.0) = 4
pow(9, 0.5) = 3
pow(2.5, -2) = 0.16
```

算术表达式



◆在C/C++程序中书写数值表达式，就是一个利用C/C++运算符将算术表达式直接转换为C/C++表达式的过程

■例如算术表达式 $\frac{3 + 4x}{5} - 10\left(\frac{4}{x} + \frac{9 + x}{y}\right)$ 可转换为如下C++表达式

$(3 + 4 * x) / 5 - 10 * (4 / x + (9 + x) / y)$

■使用数学计算规则

- 先乘除，后加减
- 括号内先计算，括号可以嵌套，最内层括号内的表达式最先运算
- 同级别按从左到右顺序



◆ 示例：将华氏温度值转换为摄氏温度值，转换公式为：

$$celsius = \frac{5}{9}(fahrenheit - 32)$$

I
P
O

```
#include <iostream>
using namespace std;

int main() {
    // Enter a degree in Fahrenheit
    double fahrenheit;
    cout << "Enter a degree in Fahrenheit: ";
    cin >> fahrenheit;

    // Obtain a celsius degree
    double celsius = (5.0 / 9) * (fahrenheit - 32);

    // Display result
    cout << "Fahrenheit " << fahrenheit << " is " <<
        celsius << " in Celsius" << endl;

    return 0;
}
```

D:\Edu-CPP\chap02\ex02-11.exe

```
Enter a degree in Fahrenheit: 60
Fahrenheit 60 is 15.5556 in Celsius
```



◆ 示例：显示当前时间

■ 算法步骤：

- (1) 通过调用ctime头文件中的time(0)函数，得到自格林尼治标准时间1970年1月1日00:00:00至当前时刻所经过的秒数totalSeconds
- (2) 计算totalSeconds % 60，得到**当前时刻的秒值**currentSecond
- (3) 计算totalSeconds / 60，得到总分钟数totalMinutes
- (4) 计算totalMinutes % 60，得到**当前时刻的分钟值**currentMinute
- (5) 计算totalMinutes / 60，得到总小时数totalHours
- (6) 计算totalHours % 24，得到**当前时刻的小时值**currentHour
- (7) 显示currentHour、currentMinute、currentSecond，即显示当前时间

I
P
O

```
#include <iostream>
#include <ctime>
using namespace std;

int main() {
    // Obtain the total seconds since the midnight, Jan 1, 1970
    int totalSeconds = time(0);

    // Compute the current second in the minute in the hour
    int currentSecond = totalSeconds % 60;
    // Obtain the total minutes
    int totalMinutes = totalSeconds / 60;
    // Compute the current minute in the hour
    int currentMinute = totalMinutes % 60;
    // Obtain the total hours
    int totalHours = totalMinutes / 60;
    // Compute the current hour
    int currentHour = totalHours % 24;
    // Display results
    cout << "Current time is " << currentHour << ":"
        << currentMinute << ":" << currentSecond << " GMT" << endl;

    return 0;
}
```

在国内，小时数应比UTC多8

D:\Edu-CPP\chap02\ex02-12.exe

Current time is 14:16:30 GMT

```
#include <iostream>
#include <ctime>
using namespace std;

int main( ) {
    time_t now = time(0);

    // 方法1: 将时间转换为字符串
    char* currentTime = ctime(&now);
    // 输出当前日期和时间
    cout << "当前日期和时间: " << currentTime << endl;

    // 方法2
    tm *ltm = localtime(&now); // 我们是+8区, 比UTC多8个小时

    // 输出 tm 结构的各个组成部分 (月是0~11, 故要加1)
    cout << "年: " << 1900 + ltm->tm_year << endl;
    cout << "月: " << 1 + ltm->tm_mon << endl;
    cout << "日: " << ltm->tm_mday << endl;
    cout << "时间: " << ltm->tm_hour << ":";
    cout << ltm->tm_min << ":";
    cout << ltm->tm_sec << endl;
    return 0;
}
```

本代码仅用于了解前面示例代码
小时数“准确化”的解决方法，
后续学习即可理解代码所用技术

D:\Edu-CPP\chap02\ex02-12B.exe

当前日期和时间: Wed Jul 30 22:19:33 2025

年: 2025

月: 7

日: 30

时间: 22:19:33



简写运算符

◆数值运算符+、-、*、/、%可以结合赋值运算符“=”形成简写运算符

■在编写时，常遇到“使用一个变量的值，对它进行修改，然后再将结果赋值给该变量”的情况，例如：

count = count + 1; // 可以简写为：

count += 1;

■其中“+=”称为“加法赋值运算符”，其它简写运算符如表所示

◆简写运算符在表达式中没有其它运算符后再执行，例如：

x /= 4 + 5.5 * 1.5; // 等同于：

x = x / (4 + 5.5 * 1.5);

运算	名字	示例	等价语句
+=	加法赋值	x += 8;	x = x + 8;
-=	减法赋值	x -= 8;	x = x - 8;
*=	乘法赋值	x *= 8;	x = x * 8;
/=	除法赋值	x /= 8;	x = x / 8;
%=	取模赋值	x %= 8;	x = x % 8;

自增、自减运算符



◆ 自增（++）、自减（--）运算符用于使变量自增或自减1

```
int i = 3, j = 3;
```

```
i++; // i的值为4
```

```
j--; // j的值为2
```

- “i++” 读作“i加加”，“j--” 读作“j减减”，其中的自增、自减运算符称为“**后自增**”、“**后自减**”（操作符在变量之后）

```
int i = 3, j = 3;
```

```
++i; // i的值为4
```

```
--j; // j的值为2
```

- “++i” 让i加1，“j--” 让j减1，其中的自增、自减运算符称为“**前自增**”、“**前自减**”（操作符在变量之前）

- i++、++i或j--、--j在前述示例中的效果是一样的，但**用在表达式中时效果不同**



◆ 表达式中的自增、自减运算符

运算	名字	描述	示例 (i初值为1)
<code>++var</code>	前自增	将var的值增加1，并使用var增加1后的新值进行后续运算	<code>int j = ++i;</code> j值为2，i值为2
<code>var++</code>	后自增	将var的值增加1，但使用原值进行运算	<code>int j = i++;</code> j值为1，i值为2
<code>--var</code>	前自减	将var的值减1，并使用var减1后的新值进行后续运算	<code>int j = --i;</code> j值为0，i值为0
<code>var--</code>	后自减	将var的值减1，但使用原值进行运算	<code>int j = i--;</code> j值为1，i值为0



◆ 自增、自减运算符示例

```
#include <iostream>
using namespace std;

int main( )
{
    int i = 10;
    int newNum = 10 * i++;
    cout << "i = " << i << ", newNum = " << newNum << endl;

    int j = 10;
    int newNum2 = 10 * (++j);
    cout << "j = " << j << ", newNum2 = " << newNum2 << endl;

    return 0;
}
```

去掉括
号后?

数值运算符的优先级

高 ++、-- 优先级相同
↓
低 +、- 优先级相同

```
int newNum = 10 * i;
i = i + 1;
```

```
j = j + 1;
int newNum2 = 10 * j;
```

D:\Edu-CPP\chap02\ex02-13.exe

```
i = 11, newNum = 100
j = 11, newNum2 = 110
```



◆ 自增、自减运算符示例

```
#include <iostream>
using namespace std;

int main( )
{
    int a = 6;
    int b = a++;
    cout << "a = " << a << ", b = " << b << endl;

    a = 6;
    b = ++a;
    cout << "a = " << a << ", b = " << b << endl;

    return 0;
}
```

 D:\Edu-CPP\chap02\ex02-14.exe

```
a = 7, b = 6
a = 7, b = 7
```



◆ 使用自增和自减运算符使表达式变得简短，但也使它们变得复杂和难以阅读。**避免**在修改**多个**变量或**多次**修改同一变量的表达式中使用这些操作符，例如：

```
int k = ++i + i;
```

本章主要内容



2.1 编写简单的程序

2.2 标识符与变量、命名常量

2.3 赋值语句与赋值表达式

2.4 数值数据类型、运算及运算符优先级

2.5 数值类型转换



2.6 软件开发流程简介

2.7 常见错误

2.8 作业与实践



混合运算时数据类型的转换

◆考虑如下语句，**i**、**k**、**d**的最后值各是什么？（**必考**）

```
short i = 100;  
long k = i * 3 + 5;  
double d = i * 3.1 + k / 2;
```

两双精度浮点数相乘，
结果为310.0

两整数相除，
结果为152

```
D:\Edu-CPP\chap02\ex02-15.exe
```

```
i = 100, k = 305, d = 462
```

◆隐式转换

■赋值时的隐式转换：将表达式的值，转换为被赋值变量的数据类型并赋值

➤例如：double d = 3; // **扩展** “3”的数据类型

■运算符两边的操作数类型一致才能进行运算，当不一致时会自动转换，转换原则：向占内存“最大”类型转换

➤例如：double celsius = (5.0 / 9) * (100 - 32);

◆显式转换（强制转换）

■避免编译警告

```
double d = 3.6;  
int i = static_cast<int>(d); // 缩小 “d”的数据类型，但d的值不变  
int k = (int)10.9;           // 缩小：小数部分被截断
```



◆ 类型转换并不改变被转换变量的值 【`int k = (int)d; // d的值不变`】

■ 示例：给出一个显示小数点后两位的销售税，税率6%

```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{
    // Enter purchase amount
    double purchaseAmount;
    cout << "Enter purchase amount: ";
    cin >> purchaseAmount;
```

```
    double tax = purchaseAmount * 0.06;
    cout << "Sales tax is " << static_cast<int>(tax * 100) / 100.0;
```

```
    return 0;
```

```
}
```

如果将100.0改为100，同样输入时的结果将如何？

The screenshot shows a C++ IDE with the following code:

```
9      cin >> purchaseAmount;
10
11     double tax = purchaseAmount * 0.06;
12     cout << "Sales tax is " << static_cast<int>(tax * 100) / 100.0;
13
14     return 0;
15 }
```

The debugger window is open, showing the following:

- Compile Log: No errors or warnings.
- Debug: Checked.
- Find Results: No results.
- Close: Button.
- Next line: Button.
- Continue: Button.
- Next instruction: Button.
- Send command to GDB: Input field.
- Value history: Shows the value of `tax` as 11.853.

A red arrow points from the text "调试跟踪显示tax=11.853" to the value 11.853 in the code.

D:\Edu-CPP\chap02\ex02-16.exe

```
Enter purchase amount: 197.55
Sales tax is 11.85
```

示例：计算给定金额的最少货币数量



◆美元种类：1美元（=100美分）， 25美分， 10美分， 5美分， 1美分

◆举例：

■ $11.66 = 11 * 1 \text{美元} + 2 * 25 \text{美分} + 1 * 10 \text{美分} + 1 * 5 \text{美分} + 1 * 1 \text{美分}$

红色数字累加和即为
最少货币数量（个数）

◆分析

■假设输入以元为单位，先将输入化作以分为单位：11.66 美元 = 1166美分

■将其除以100，商（个数），余数（剩余的美分）： $1166 / 100 = 11 \dots 66$

■将余数以25，商（个数），余数（剩余的美分）： $66 / 25 = 2 \dots 16$

■将余数以10，商（个数），余数（剩余的美分）： $16 / 10 = 1 \dots 6$

■将余数以5，商（个数），余数（剩余的美分）： $6 / 5 = 1 \dots 1$

◆实现

■商（美元个数）：整数相除；余数（剩余的美分）：整数取余

➤ $1166 / 100$; $1166 \% 100$;

■

```

#include <iostream>
using namespace std;

int main()
{
    // Receive the amount
    cout << "Enter an amount in double, for example 11.56: ";
    double amount;
    cin >> amount;

    int remainingAmount = static_cast<int>(amount * 100);

    // Find the number of one dollars
    int numberOfOneDollars = remainingAmount / 100;
    remainingAmount = remainingAmount % 100;

    // Find the number of quarters in the remaining amount
    int numberOfQuarters = remainingAmount / 25;
    remainingAmount = remainingAmount % 25;

    // Find the number of dimes in the remaining amount
    int numberOfDimes = remainingAmount / 10;
    remainingAmount = remainingAmount % 10;

```

强制转换为整数，
会导致结果错误

```

// Find the number of nickels in the remaining amount
int numberOfNickels = remainingAmount / 5;
remainingAmount = remainingAmount % 5;

// Find the number of pennies in the remaining amount
int numberOfPennies = remainingAmount;

// Display results
cout << "Your amount " << amount << " consists of" << endl
<<
    " " << numberOfOneDollars << " dollars" << endl <<
    " " << numberOfQuarters << " quarters" << endl <<
    " " << numberOfDimes << " dimes" << endl <<
    " " << numberOfNickels << " nickels" << endl <<
    " " << numberOfPennies << " pennies";

```

8	double amount;	
9	cin >> amount;	amount = 11.56
10		
11	int remainingAmount = static_cast<int>(amount * 100);	e, for
Your amount 11.56 consists of		
1	double amount;	
2	cin >> amount;	amount = 10.029999999999999
3		
4	int remainingAmount = static_cast<int>(amount * 100);	
5	1 nickels	0 nickels
6	1 pennies	2 pennies

本章主要内容



2.1 编写简单的程序

2.2 标识符与变量、命名常量

2.3 赋值语句与赋值表达式

2.4 数值数据类型、运算及运算符优先级

2.5 数值类型转换

2.6 软件开发流程简介

2.7 常见错误

2.8 作业与实践

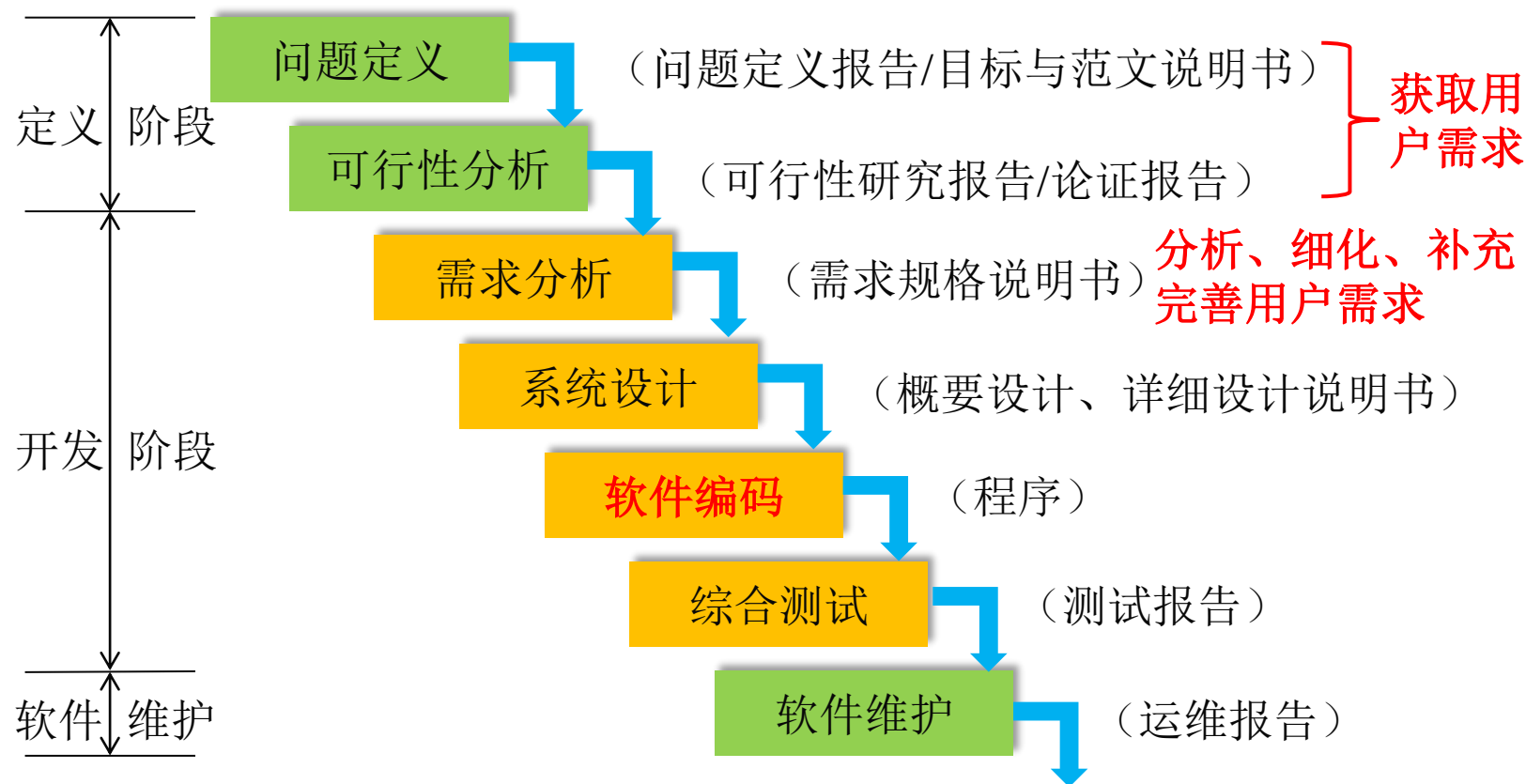


软件生命周期



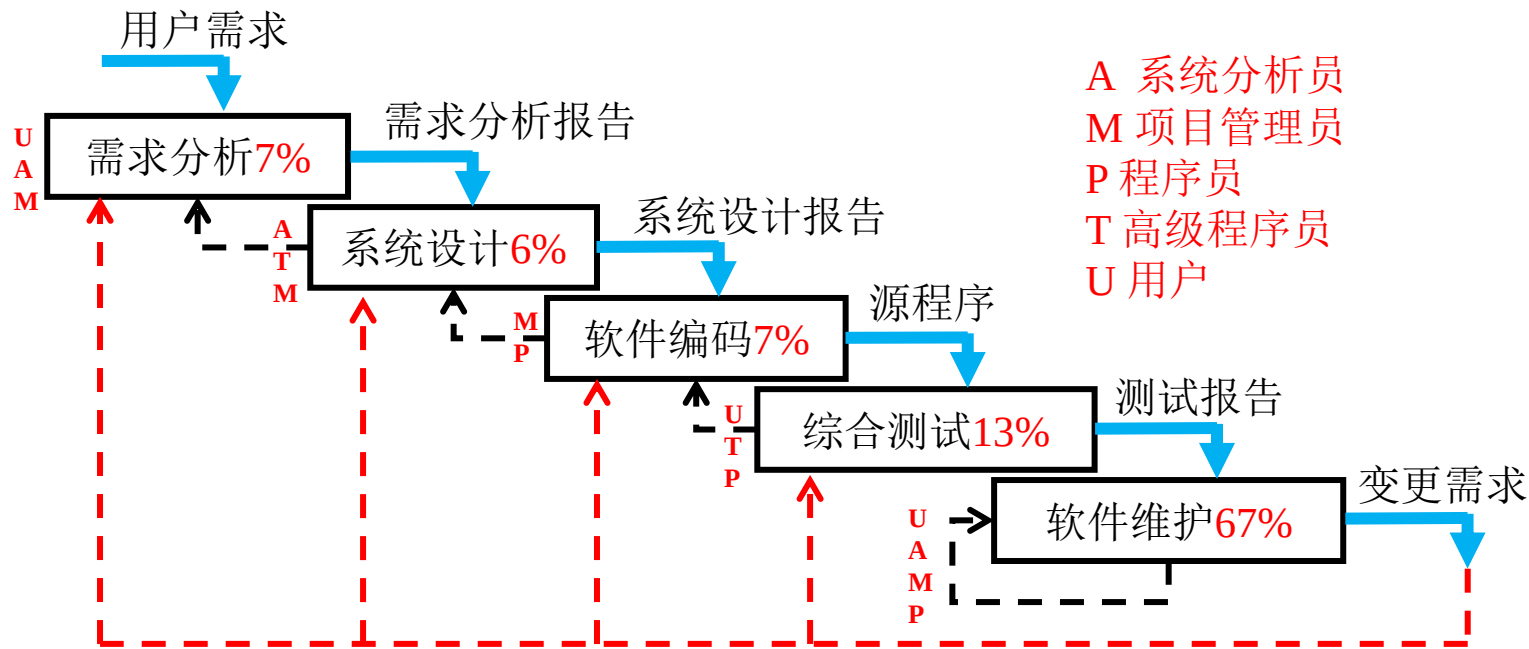
◆ 每个软件产品，无论大小，都有相同的生命周期：需求分析、系统设计、软件编码、软件测试、软件维护

软件开发过程瀑布模型，《软件工程概论》讲授





- ◆ 编程语言学习过程中给出的例子都非常简单，其需求十分明确，但实际生产中的软件需求，往往需要跟用户反复沟通，开发过程中还有可能变更需求、完善需求等各种“返工”



本章主要内容



2.1 编写简单的程序

2.2 标识符与变量、命名常量

2.3 赋值语句与赋值表达式

2.4 数值数据类型、运算及运算符优先级

2.5 数值类型转换

2.6 软件开发流程简介

2.7 常见错误

2.8 作业与实践





◆ 常见错误1：未定义/未初始化变量和未使用变量

```
double interestRate;  
double interest = interestrates * 45;
```

◆ 常见错误2：整数溢出

```
short y = 32767 + 1;
```

◆ 常见错误3：四舍五入错误

```
float a = 1000.43;  
float b = 1000.0;  
cout << a - b << endl; // 0.429993
```

◆ 常见错误4：意外的整数除法

```
double d = 5 / 9;
```

◆ 常见错误5：忘记头文件

```
#include <iostream>  
using namespace std;
```

```
int main()
```

```
{
```

```
    short int x, y;
```

```
    unsigned short int z;
```

```
    x = 32767;
```

```
    y = x + 1;
```

```
    z = x + 1;
```

```
    cout << "y = " << y << ", z = " << z << endl;
```

```
    return 0;
```

```
}
```

x = 32767

0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

+1

1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

D:\Edu-CPP\chap02\ex02-18.exe

y = -32768, z = 32768

本章主要内容



2.1 编写简单的程序

2.2 标识符与变量、命名常量

2.3 赋值语句与赋值表达式

2.4 数值数据类型、运算及运算符优先级

2.5 数值类型转换

2.6 软件开发流程简介

2.7 常见错误

2.8 作业与实践



第02章作业



- (1)编写程序读入一个double型的摄氏温度值，将其转化为华氏温度值并显示结果。转换公式为：
$$\text{fahrenheit} = (9 / 5) * \text{celsius} + 32$$
- (2)编写程序读取一个圆柱体的半径和长度，计算并显示其表面积和体积
- (3)编写程序读入一个以英尺为单位的长度值，将其转换为以米为单位的长度值，并输出结果。1英尺等于0.305米。
- (4)编写程序读入消费小计（如¥100元）和消费费率（如5%），计算并显示小费金额和消费总计
- (5)编写程序读入一个三位整数，计算并输出其三个数字之和（如输入234，输出9）
- (6)编写程序读入秒数，计算秒数对应的时分秒（如读入3700秒，输出“1小时1分40秒”）



(7)编写程序提示用户输入以米/秒为单位的初始速度 v_0 、末速度 v_1 、所用时间秒数 t ，利用公式“ $a = (v_1 - v_0) / t$ ”计算并显示加速度

(8)编写程序提示用户输入两个点坐标 (x_1, y_1) 和 (x_2, y_2) ，输出连接此两点的直线斜率。斜率公式为： $(y_2 - y_1) / (x_2 - x_1)$

(9)编写程序提示用户输入驾驶公里数、以公里升为单位的汽车耗油效率、每升的价格，输出这趟路程的花费

(10)编写程序输出下面的列表

➤ 后续课程学习了循环语句，再优化该题的编程

a	b	pow(a, b)
1	2	1
2	3	8
3	4	81
4	5	1024